LA Food Scout

A Machine Learning App for Discovering Highly Rated Restaurants in Los Angeles

Presenter: Zhixian Wang
Course: STAT 418 Final Project Proposal

Project Overview & Motivation

Brief description: A machine learning app that predicts whether an LA restaurant is highly rated (≥4.5★) using only its metadata — price, cuisine type, and neighborhood

Problem statement: With 25,000+ restaurants in LA, diners waste time on overhyped spots. New restaurants with few reviews are especially hard to evaluate — there's no reliable signal yet

Motivation: Yelp/Google ratings are powerful but lag behind reality. A model trained on metadata can surface hidden gems before they accumulate hundreds of reviews. I picked this project because discovering great local food is something every UCLA student deals with

Target users: LA residents, UCLA students, and tourists who want smarter, faster dining decisions

Data Collection Approach & Progress

Data source: Google Places API (New)

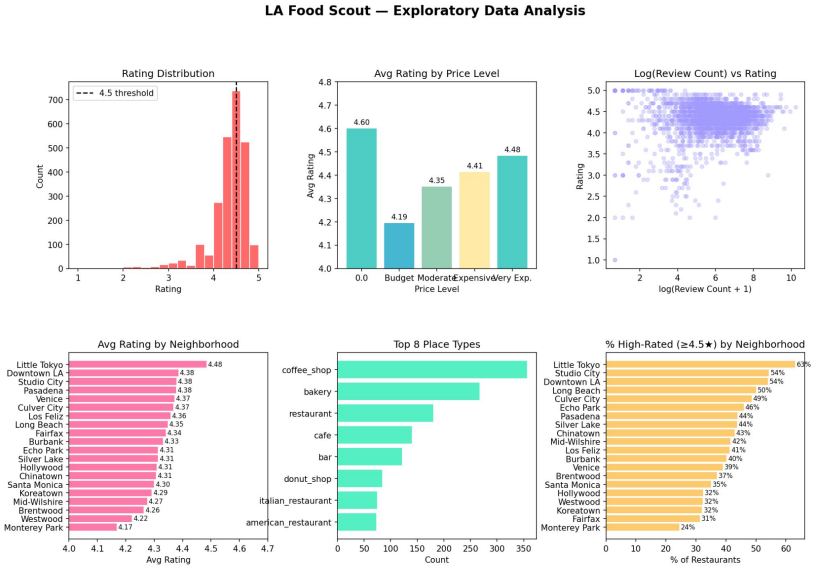
Collection method: Python script using requests + python-dotenv(API). Searched by place type across LA neighborhoods with 5 offset search points per neighborhood to overcome the 20-result-per-call cap.

Current progress:

- 2,443 unique rated restaurants collected ✅
- 20 LA neighborhoods covered

name	rating	review_count	price_level	primary_type	neighborhood
Howlin' Ray's	4.7	3821	2	restaurant	Chinatown
Gjusta	4.6	5102	2	bakery	Venice
Sushi Gen	4.5	2943	2	restaurant	Little Tokyo
In-N-Out Burger	4.4	8201	1	restaurant	Hollywood
Nobu	4.3	1872	4	restaurant	Brentwood

Exploratory Data Analysis (EDA)



Patterns discovered:

- Ratings cluster tightly between 4.0–4.7 — very little below 3.5 (only 4.1%)
- Price level has a clear positive effect: Budget avg = 4.19, Very Expensive avg = 4.48
- Little Tokyo (4.48 avg) and Studio City (4.38) are top neighborhoods; Monterey Park (4.17) and Hollywood (4.23) trail
- 40.9% of restaurants qualify as high-rated (≥4.5★)

Data quality issues:

- 637 restaurants (26%) are missing price_level → will impute with neighborhood + cuisine type median
- 80 restaurants have no rating → already dropped from dataset
- Some business types are noisy ("donut_shop", "american_restaurant") → will consolidate into top 10 categories

Feature engineering ideas:

- log(review_count) — log-transform to reduce skew from viral spots with 10,000+ reviews
- is_budget — binary flag for price level 1
- neighborhood_avg_rating — encode neighborhood quality as a continuous feature
- review_density = review_count / (days since listed) — proxy for popularity growth rate (if timestamp available)

Proposed Modeling Approach

Model	Role	Why
Logistic Regression	Baseline	Fast, interpretable, easy to explain
Random Forest	Primary candidate	Handles non-linear patterns, robust to outliers
XGBoost	Performance benchmark	Best accuracy on tabular data in practice

Target variable: high_rated — binary (1 if rating ≥ 4.5, 0 otherwise)

Features (model inputs):

- price_level (ordinal 1–4)
- log_review_count (continuous)
- primary_type (one-hot: restaurant / cafe / bakery / bar)
- neighborhood (label encoded, 20 categories)
- is_operational (binary)
- neighborhood_avg_rating (engineered continuous feature)

Evaluation metrics:

- F1-score (primary) — balances precision and recall for the 41/59 class split
- ROC-AUC — measures overall ranking ability
- Accuracy — secondary, for easy communication
- Success criteria: F1 ≥ 0.75 on 20% held-out test set

Baseline approach: Logistic Regression trained on price level + review count only — anything above this means the additional features add real value

Timeline & Next Steps

Current status:

- ✅ Data collected — 2,443 restaurants, 20 neighborhoods
- ✅ Initial EDA complete — 6 charts generated
- ✅ Project GitHub repo created
- 🔄 Feature engineering in progress

Week	Milstone	Status
Week 6	Data collection + proposal presentation	✅ Done
Week 7	Full EDA, feature engineering, train Logistic Regression baseline	⬜
Week 8	Train Random Forest + XGBoost, SHAP analysis, pick final model	⬜
Week 9	Build FastAPI endpoint, deploy to Google Cloud Run	⬜
Week 10	Build + deploy Streamlit app, end-to-end testing, final presentation	⬜

Challenge	Mitigation
26% missing price level	Impute with neighborhood + type median
Ratings skew high (most 4.0–4.7)	Use 4.5 threshold; tune decision boundary
Google Cloud Run deployment complexity	Use Streamlit Community Cloud as fallback (free, no Docker needed)
Model F1 below 0.75	Use other models as backup

Remaining work: feature engineering → model training → API → Streamlit app → deployment